

Potato

- Ժամանակի սահմանափակում՝ 10 րոպե
- Միջավայր՝ մեկ GPU (≈16 GB VRAM), առանց ինտերնետի
- Լուծման չափ՝ solution.ipynb ≤ 1 ՄԲ
- Հիշողություն՝ 5 ԳԲ

Առաջադրանք

Ձեր ընկերն առաջարկում է խաղալ գուշակության խաղ: Նա, որպես մրցավար, ֆիքսված բառապաշարից ընտրում է մեկ թաքնված բառ, իսկ դուք պետք է գտնեք այն առավելագույնը 30 քայլում: Յուրաքանչյուր քայլի մրցավարը համեմատում է երկու բառ և հայտնում, թե որն է իմաստով ավելի մոտ թաքնված բառին: Յուրաքանչյուր խաղ սկսվում է ֆիքսված `lamp vs potato` զույգից, քանի որ դրանք ձեր ընկերոջ սիրելի իրերից երկուսն են: Այնուհետև ձեր ծրագիրը առաջարկում է մեկ նոր բառ: Համեմատության հաղթողը պահպանվում է և համեմատվում ձեր հաջորդ առաջարկի հետ: Դուք հաղթում եք խաղը այն պահին, երբ ճիշտ առաջարկում եք թաքնված բառը: Համընկնումը հաշվի չի առնում մեծատառ/փոքրատառ տարբերությունը: Ձեր առաջարկած յուրաքանչյուր բառ պետք է լինի `dataset/vocabulary.json`-ում:

`solution.ipynb`-ում առկա է ամբողջական օրինակ՝ խաղի արձանագրությամբ (protocol) և տվյալների բեռնմամբ: Դուք կարող եք փոփոխել `PublicEmbeddingPlayer` դասը: Ձեր ծրագիրը ինիցիալիզացվում է մեկ անգամ և խաղում է բոլոր խաղերը ֆայլի մեկ աշխատանքի ընթացքում. արձանագրությունը յուրաքանչյուր խաղի սկզբում ստեղծում է նոր `PublicEmbeddingPlayer`:

Մրցավարը

Ձեր ծրագիրը ուղարկում է մեկ JSON օբյեկտ Մրցավարին, իսկ Մրցավարը պատասխանում է մեկ JSON օբյեկտով:

խաղի մի օրինակ, որտեղ թաքնված բառը ցուցադրված է միայն այս արձանագրությունը (protocol) ձեզ ցույց տալու համար.

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}          <- matches: game won
-> {"status": "win"}
```

Քայլերը ինդեքսավորվում են 1-ից 30-ը:

`verdict`-ի տարբերակներն են՝ `first`, ինչը նշանակում է, որ `word1`-ն ավելի մոտ է: Կամ `second`, ինչը նշանակում է, որ `word2`-ն ավելի մոտ է, կամ `same`, ինչը նշանակում է, որ երկու բառերն էլ հավասարապես մոտ են թաքնված բառին:

`winner_word`-ը հաջորդ համեմատության համար պահպանված բառն է: `same` վճռի դեպքում առաջին բառը մնում է:

Տվյալների հավաքածու

Ընդհանուր է բոլոր բաժանումների (splits) համար.

- `dataset/vocabulary.json` — 1602 եզակի փոքրատառ բառեր: Թաքնված բառը միշտ սրանցից մեկն է:
- `dataset/public_embeddings.npy` — `float32`, չափսը՝ (1602, 2560): `i` տողը համապատասխանում է բառարանի `i` բառին: Սրանք *հանրային* embedding-ներ են. մրցավարն օգտագործում է այլ՝ մասնավոր ներկայացում (embedding):

Բաժանումները (split) թաքնված բառերի բազմություններ են.

Split	Բառեր	Պատասխաններ	Օգտագործեք այն հետևյալի համար
test_public	120	<code>dataset/test_public.json</code>	ձեր լուծումը աշխատեցնելու և ինքնազնահատվելու համար
test_leaderboard_a	120	թաքնված	live leaderboard
test_leaderboard_b	120	թաքնված	final ranking

train բաժանում չկա — նշանագրված տողերից ոչինչ չի վարժեցվում (nothing is fitted from labelled rows):

Տրամադրված մոդելներ

Առաջադրանքի հետ տրամադրվում են երկու նախապես վարժեցված embedding մոդելներ, որոնք կարող են օգտագործվել.

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/iaoi/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/iaoi/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Երկուսն էլ պետք է բեռնվեն իրենց local path-ից: Hugging Face-ից hub ID, օրինակ "BAAI/bge-m3"-ը ծախողվեցու է, քանի որ գնահատումն իրականացվում է offline: Յուրաքանչյուր թղթապանակ պարունակում է աշխատող example.py, որը ցույց է տալիս offline կանչը:

Հասանելի գրադարաններ՝ numpy, torch, sentence-transformers: Առանց ինտերնետի, առանց ներբեռնումների, առանց այլ փաթեթների:

Output

Չկա: Սա ինտերակտիվ առաջադրանք է. ձեր լուծումը պատասխանի ֆայլ չի գրում. այն հաղորդակցվում է մրցավարի հետ stdin/stdout-ի միջոցով, ինչպես նկարագրված է վերևում:

Մետրիկա

t քայլում գտնված խաղը տալիս է $1.0 - 0.02 \times \max(0, t - 10)$ միավոր, իսկ 30 քայլի ընթացքում չլուծված խաղը տալիս է 0 միավոր: Այսպիսով, 1-10 քայլերը տալիս են 1.00, 20-րդ քայլը տալիս է 0.80, 30-րդ քայլը տալիս է 0.60:

Առաջադրանքից ձեր միավորը խաղերի միջին միավորն է $\times 100$ ՝ 0.00-ից 100.00 միջակայքում:

10 րոպեի սահմանափակումը ներառում է գործարկումը, նախապատրաստումը և թեստային հավաքածուի բոլոր 120 խաղերը:

Ինչպես ուղարկել

- Բացեք solution.ipynb-ը, խմբագրեք PublicEmbeddingPlayer-ը և աշխատեցրեք բոլոր cell-երը՝ համոզվելու համար, որ այն աշխատում է:
- Ըստ ցանկության, ստուգեք այն locally՝ `python local_test.py solution.ipynb --limit 5`: Local մրցավարն օգտագործում է *հանրային* embedding-ները, ուստի դրա միավորը միայն կողմնորոշիչ է:
- Պահպանեք solution.ipynb-ը:
- Բացեք Git ներդիրը JupyterLab-ի ձախ կողմնային վահանակում:
- Stage արեք solution.ipynb-ը (դրա կոդքի + պատկերակը):
- Մուտքագրեք commit հաղորդագրությունը և սեղմեք Commit:
- Սեղմեք դեպի վեր սլաքով ամպի պատկերակը՝ push անելու համար:
- Վերադարձեք այս Մրցույթի էջ և սեղմեք Submit՝ ձեր տրամադրած հաղորդագրությանը համապատասխանող commit հաղորդագրությամբ:

Ուղարկեք ճշգրիտ մեկ ֆայլ՝ solution.ipynb անունով, որը ներառում է ցանկացած անհրաժեշտ նախապատրաստություն և ինֆերենս: